



Google Cloud & Vertex AI 기반 스타트업 AI 비서 서비스 개발 실전



시간	모듈	세부 내용
09:00~10:00	GCP 환경 세팅 / API 연동	[인프라] Google Cloud 프로젝트 생성, Vertex AI API 활성화, API Key 발급 실습
10:00~11:30	나만의 지식 베이스 구축 (RAG)	[데이터] 기업 문서 업로드, Cloud Storage 연동, RAG 원리 이해 및 실습
11:30~12:30	실무 맞춤형 프롬프트 엔지니어링	[기획] AI 페르소나 설계, Tone & Manner 부여, 환각 방지 프롬프트 제어
12:30~13:30	점심시간	
13:30~15:30	Streamlit 활용 웹 UI 구현	[개발] 챗봇 대화창 레이아웃 구성, Gemini 연동 라이브 코딩 실습
15:30~17:00	Cloud Run 초고속 배포	[배포] 서버리스 런칭, 공개 URL 생성 및 모바일 구동 확인
17:00~18:00	데모데이	[발표] 5종 AI 비서 시연, 강사 피드백, 네트워킹, 수료증 수여

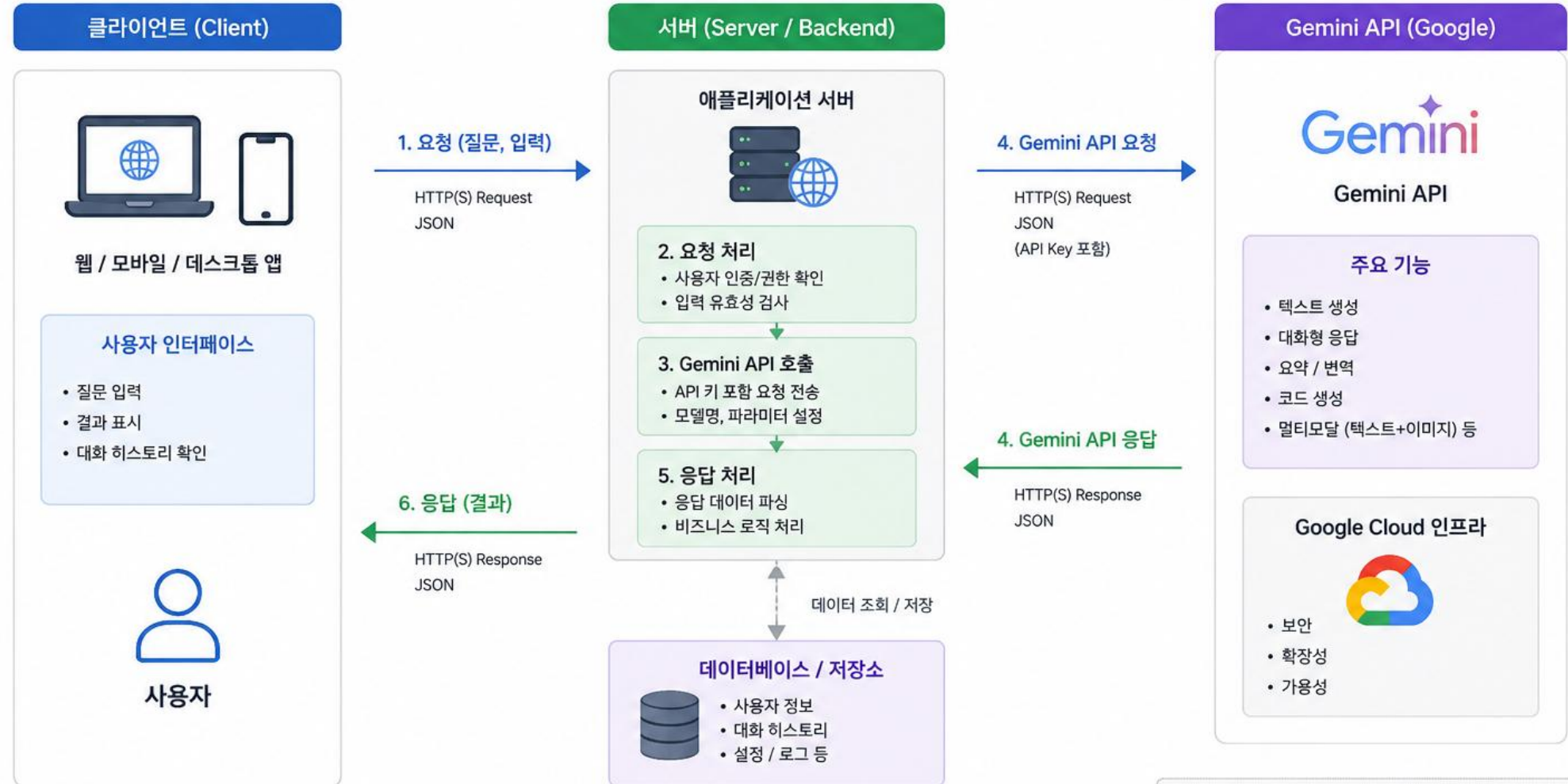


개발환경

- ▶ Antigravity ide: <https://antigravity.google/download?app=antigravity-ide>
- ▶ 파이썬: python 12.09
- ▶ 필수 설치 라이브러리: requirements.txt

Gemini API 사용 전체 구조 (클라이언트 - 서버)

사용자가 클라이언트를 통해 요청을 보내면, 서버가 Gemini API를 호출하고 응답을 받아 사용자에게 전달합니다.



흐름 요약

- 1 사용자 클라이언트에서 질문을 입력
- 2 서버가 요청을 받고 결중 및 처리
- 3 서버가 Gemini API로 요청을 전송
- 4 Gemini API가 응답을 생성하여 서버로 반환
- 5 서버가 응답을 가공하여 클라이언트로 전달
- 6 사용자에게 결과가 표시됨

필수 구성 요소

- Gemini API Key (Google AI Studio 발급)
- HTTPS 통신 (보안)
- 백엔드 서버 (API 호출 및 응답 처리)

Vertex AI란?

ML 개발의 모든 단계를 하나로 통합한 Google Cloud의 **완전관리형 AI 플랫폼**



Vertex AI는 데이터 준비부터 모델 학습, 배포, 예측, 모니터링까지 머신러닝 워크플로우의 전체 라이프사이클을 지원합니다.

1. 데이터 소스

- Cloud Storage (파일, 이미지, 동영상 등)
- BigQuery (구조화 데이터)
- Pub/Sub, Dataflow (실시간 스트리밍 데이터)
- Cloud SQL, Spanner 등 (데이터베이스)
- 온프레미스 데이터 (Cloud VPN / Interconnect)

2. Vertex AI – 통합 ML 플랫폼

데이터 준비

- 데이터 라벨링 (AutoML Data Labeling)
- 데이터셋 관리 (Feature Store)

모델 개발

- AutoML (자동 모델 생성)
- 커스텀 학습 (TensorFlow, XGBoost 등)
- 분산 학습 지원

모델 관리

- 모델 버전 관리
- 모델 레지스트리
- 실험 추적 (Experiments)

모델 배포

- 엔드포인트 배포 (온라인 예측)
- 배치 예측
- 스케일 자동 조정

모니터링 & 관리

- 성능 모니터링
- 데이터 드리프트 감지
- 모델 성능 검증

공통 기반

- IAM & 보안
- VPC Service Controls
- 암호화
- 로깅 & 감사
- 규정 준수

3. 활용 (예측 & 운영)

- 실시간 예측 서비스 (웹/모바일 앱, API)
- 배치 예측 결과 (보고서, 대시보드)
- 비즈니스 인사이트 (의사결정 지원)
- 앱 및 서비스 통합 (Workflow, Chatbot 등)

Vertex AI의 주요 장점



완전관리형 서비스
인프라 관리 없이 ML에 집중



개발 속도 향상
AutoML, 도구 통합으로 빠른 모델 개발



비용 최적화
필요한 리소스만 사용, 자동 스케일링



보안 & 거버넌스
기업 수준의 보안, 컴플라이언스 지원



확장성
대규모 데이터와 트래픽도 안정적 처리



Google Cloud 생태계 연동
BigQuery, Dataflow, Looker 등과 자연스럽게 통합



활용 예시

고객 이탈 예측

수요 예측

이미지/문서 분석

챗봇/상담 자동화

이상 탐지

추천 시스템

...and more!

Google Cloud Storage란?

Google Cloud Storage(GCS)는 Google Cloud에서 제공하는 확장성, 내구성, 가용성이 높은 객체 스토리지 서비스입니다.



1. 주요 특징



높은 내구성과 가용성

데이터는 여러 위치에 중복 저장되어 99.999999999%(11 9s)의 내구성을 제공합니다.



무제한 확장성

데이터 양에 제한 없이 페타바이트(PB) 단위까지 저장할 수 있습니다.



보안 및 접근 제어

IAM, 암호화, 버킷 정책 등을 통해 안전하게 데이터를 보호하고 세밀한 접근 제어가 가능합니다.



비용 효율성

사용한 만큼만 지불하며, 다양한 스토리지 클래스 선택으로 비용을 최적화할 수 있습니다.

2. 구성 요소



버킷 (Bucket)

데이터를 담는 최상위 컨테이너

- 전역적으로 고유한 이름
- 리전 또는 멀티 리전 위치 선택



객체 (Object)

버킷에 저장되는 개별 데이터

- 파일과 메타데이터로 구성
- 고유한 이름(키)으로 식별

예시



my-bucket
(버킷 이름)

- images/photo.jpg
- videos/intro.mp4
- docs/report.pdf
- backup/database.sql
- ...

객체 구성

구성 요소	설명
데이터	실제 파일 내용
메타데이터	크기, 유형, 생성 시간 등
객체 이름(키)	버킷 내에서 객체를 식별하는 고유 경로

3. 스토리지 클래스



Standard

자주 액세스하는 데이터에 적합
(높은 가용성, 짧은 지연 시간)



Nearline

월 1회 정도 액세스하는 데이터에 적합
(저비용, 약간 긴 검색 시간)



Coldline

분기별 1회 정도 액세스하는 데이터에 적합
(더 낮은 비용, 긴 검색 시간)



Archive

연 1회 미만 액세스하는 데이터에 적합
(가장 낮은 비용, 긴 검색 시간)

4. 활용 사례



백업 및 재해 복구

중요 데이터를 안전하게 백업하고 재해 발생 시 빠르게 복구



미디어 저장 및 스트리밍

대용량 이미지, 동영상 파일을 저장하고 전 세계 사용자에게 제공



데이터 레이크 및 분석

빅데이터를 저장하고 분석/ML 워크로드에 활용



정적 웹사이트 호스팅

정적 웹사이트 파일을 저장하고 전 세계에 빠르게 배포



장기 보관

규정 준수, 기록 보관, 아카이브 데이터를 장기 보관

5. 주요 연동 서비스



Compute Engine



BigQuery



Cloud Functions



Dataflow



Cloud Dataproc

...and more

실무 맞춤형 프롬프트 엔지니어링

[기획] AI 페르소나 설계, Tone & Manner 부여, 환각 방지 프롬프트 제어



1. AI 페르소나(Persona) 설계

AI에게 어떤 역할(전문가)을 수행할지 정의하여 답변의 전문성과 일관성을 높이는 과정

왜 필요한가?

페르소나 없음



일반적인 답변

VS

페르소나 있음



전문가처럼 사고하고 답변

좋은 페르소나 구성 요소



예시

- 당신은 Python 데이터 분석 전문가
- 실무 경력 15년
- Pandas, NumPy, Matplotlib 전문
- 초보 개발자를 교육하는 강사
- 이해하기 쉽게 설명한다.



2. Tone & Manner 부여

같은 내용을 어떤 말투(Tone)와 형식(Manner)으로 전달할지 정의하여 원하는 스타일의 답변을 유도

Tone (말투)



Manner (답변 스타일)



예시 프롬프트

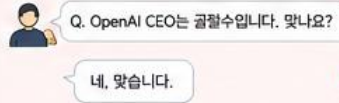
- 당신은 Python 강사입니다. 초보자도 이해할 수 있도록
- 쉬운 용어 사용
 - 단계별 설명
 - 코드 예시 포함
 - 마지막에 핵심 요약 작성

3. 환각(Hallucination) 방지

LLM이 없는 사실을 그럴듯하게 만들어내는 현상(환각)을 줄이고, 사실 기반의 신뢰성 높은 답변을 유도

환각이란?

LLM이 없는 사실을 그럴듯하게 만들어내는 현상



환각 방지 기법 7가지

1. 모르면 모른다고 답하기
"모르는 내용은 추측하지 않고 '알 수 없습니다.'라고 답하세요."
2. 근거 제시
"답변에 근거를 함께 작성하세요."
3. 추론과 사실 구분
"사실과 추론을 구분하여 작성하세요."
4. 출처 표시
"가능한 출처를 함께 제시하세요."
5. 불확실성 표현
"확실하지 않다면 '가능성이 있습니다.'라고 표현하세요."
6. 범위 제한
"답변할 문서 안에서만 답변하세요."
7. 단계별 검증
"답변을 작성한 후 오류 여부를 다시 검토하세요."

4. 실무에서 많이 사용하는 프롬프트 구조



구조 예시

역할	당신은 Python 교육 전문가입니다.
목표	목표는 초보자가 Pandas를 이해하도록 돕는 것입니다.
대상	대상은 비전공자입니다.
Tone	친절하고 쉬운 말투로 설명하세요.
Manner	단계별 설명, 예시 포함, 코드 포함, 마지막에 핵심 요약 작성
제약조건	어려운 용어 사용 금지, 모르는 내용은 추측하지 않음
출력 형식	1. 개념 2. 예제 3. 실습 4. 핵심 요약

5. 종합 예제 프롬프트

당신은 10년 경력의 AI 교육 전문가입니다.

목표는 학생을 AI를 처음 배우는 초보자에게 프롬프트 엔지니어링을 쉽게 설명하는 것입니다.

답변은 친절하고 전문적인 Tone으로 작성하세요.

모든 설명은

- 단계별
- 표 포함
- 실무 예제 포함
- 마지막 핵심 요약 작성

제약조건

- ✓ 어려운 용어는 쉬운 말로 설명
- ✓ 추측하지 말 것
- ✓ 불확실하지 않은 내용은 불확실하다고 명시
- ✓ 공식 문서에 근거한 내용 우선
- ✓ 사실과 의견을 구분하여 작성

출력 형식

1. 개념
2. 원리
3. 실무 예시
4. 주의사항
5. 핵심 요약

핵심 요약

구성 요소	목적	핵심 내용	실무 활용 예시
AI 페르소나 (Persona)	AI의 역할과 전문성을 정의	역할, 경력, 전문 분야, 목표, 제약조건을 설정하여 일관된 답변 생성	15년 경력 데이터 분석가 - 초보자를 가르치는 Python 강사
Tone & Manner	답변의 말투와 표현 방식 통일	Tone은 말투(친절, 공식적), Manner는 형식(표, 단계별, 예시, 요약)	- 교육 자료, 고객 응대 - 기술 문서 작성
환각(Hallucination) 방지	사실 기반의 신뢰성 확보	모르면 모른다고 답하기, 근거-출처 제시, 사실과 추론 구분, 범위 제한(RAG), 불확실성 명시	- 사내 문서 질의응답 - 법률·의료·금융 보조 - 정책 안내
실무 프롬프트 구조	재사용 가능한 프롬프트 설계	역할 → 목표 → 대상 → Tone → Manner → 제약조건 → 출력 형식	- GPTs, AI 에이전트 - 업무 자동화, 교육 콘텐츠 제작

★ 실무 팁

프롬프트의 품질은 모델 자체보다 역할(Persona), 표현 방식(Tone & Manner), 제약조건(환각 방지), 출력 형식(Output Format)을 얼마나 구체적으로 설계했는지에 크게 좌우된다.





API 없이 프롬프트 결과 교차 검증 방법

여러 LLM(예: ChatGPT, Claude, Gemini) 또는 하나의 LLM에서 프롬프트를 활용하여
답변의 정확성과 신뢰도를 높이는 방법



1 동일 질문을 여러 LLM에 입력

같은 질문을 여러 LLM에 입력하여
답변을 비교합니다.

예시 질문

RNN과 LSTM의 차이를 설명하시오.

결과 비교 예시

항목	ChatGPT	Claude	Gemini
가을기 소설	○	○	○
게이트 설명	○	○	○
장기 기억 설명	○	○	○

판단 기준

- 모두 동일 → 신뢰도 높음
- 하나만 다름 → 추가 확인 필요



2 반대 의견 요청

답변의 반론이나 오류 가능성을
요청하여 검증합니다.

1단계: 답변 요청

RNN과 LSTM의 차이를 설명하시오.

2단계: 반대 의견 요청 (예시)

방금 답변에서 틀렸을 가능성이 있는
부분을 찾아라.
또는
반론을 제시하라.

Tip

당신은 엄격한 AI 검토자이다.
다음 답변의 오류, 누락, 과장된 표현을 찾아라.

3 교수 + 리뷰어 방식

답변 생성 후, 교수(리뷰어) 역할로
채점하고 평가합니다.

1단계: 답변 생성

RNN과 LSTM 차이를 설명하라.

2단계: 교수 역할로 평가 요청 (예시)

당신은 대학원 교수이다.
위 설명을 채점하라.

- 정확성:
- 완전성:
- 오류:
- 누락:



4 사실 검증 전용 프롬프트

답변의 사실 여부, 근거, 신뢰도를
체계적으로 검증합니다.

다음 답변을 검증하라.

- 사실인지 확인
- 근거가 부족한 부분 표시
- 추정인지 사실인지 구분
- 신뢰도 0~100점 평가

[답변]

...

결과 예시

항목1 : 사실
항목2 : 근거 부족
...
신뢰도 : 82점



실무 추천 전체 흐름



1. 질문 입력

주제와 요구사항 명확히 설정



2. 답변 생성

LLM에 질문하여 초안 작성



3. 비판적 검토

오류, 누락, 과장 여부 확인



4. 사실 검증

근거, 출처, 최신성 확인



5. 최종 수정

검증 결과 반영하여 수정



신뢰도 높은 최종 답변

5 출처 강제

답변을 처음부터 주장, 근거, 출처 형태로
체계적으로 요청합니다.

프롬프트 예시

답변시 반드시 다음 구조로 작성하라.

- 주장
- 근거
- 출처

답변 예시

주장: LSTM은 장기 의존성 문제를 개선한다.
근거: Forget Gate 사용
출처: Hochreiter & Schmidhuber (1997)



☆ 출처가 없는 답변은 의심!

6 AI 배심원(Jury) 방식

여러 전문가 역할로 다양한 의견을 제시하고
합의안을 도출합니다.

프롬프트 예시

다음 주제에 대해 전문가A 의견
전문가B 의견
전문가C 의견
전문가D 의견
을 제시하고
마지막에 합의안을 작성하라.



출력 예시

전문가A: 필요하다
전문가B: 필요하지 않다
전문가C: 상황에 따라 다르다
합의안:
...



7 검증 전문가 프롬프트 (추천)

답변을 종합적으로 분석하고, 수정된 최종 답변까지
작성하도록 요청합니다.

프롬프트 예시

당신은 검증 전문가이다.
아래 답변을 분석하라.

- 사실 오류
- 논리 오류
- 누락된 내용
- 최신 정보 여부
- 신뢰도 점수(0~100)

그리고 수정된 최종 답변을 작성하라.

[답변]

...



8 강의용 추천 패턴

학생들이 쉽게 따라할 수 있는 검증 3단계

여러 LLM을 사용하는 경우



하나의 LLM만 사용하는 경우



생성 → 비판 → 검증 → 수정
4단계 체인이 가장 효과적!



추가 팁

- 중요한 내용은 반드시 여러 번 교차 검증하세요.
- 특히 최신 정보, 통계, 기술 내용은 사실 검증이 필수입니다.
- 출처 확인이 어렵다면 "근거 부족"으로 표시하고 보수적으로 판단하세요.



주의사항

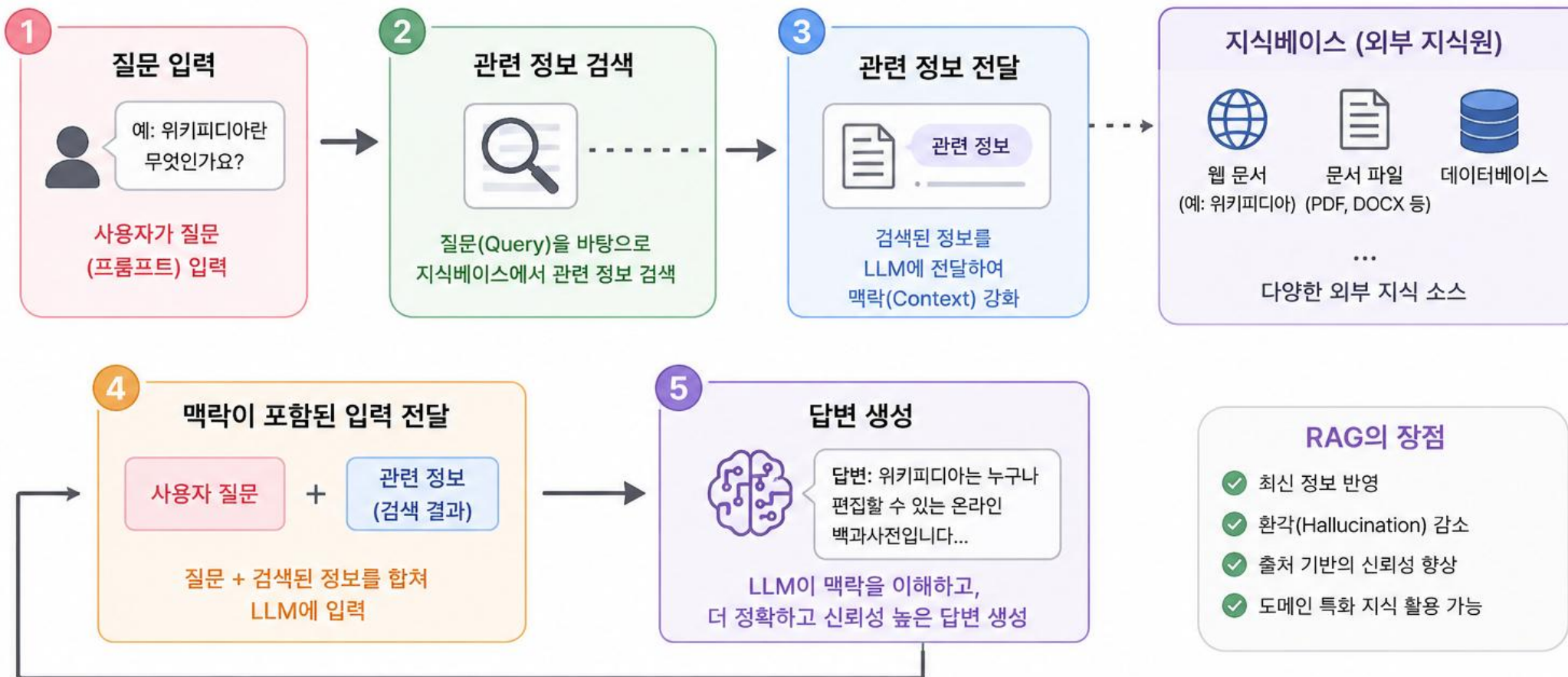
- LLM도 틀릴 수 있습니다.
- 100% 신뢰하지 말고 비판적으로 활용하세요.
- 중요한 결정은 반드시 추가적인 사실 확인이 필요합니다.

RAG(Retrieval-Augmented Generation)는 대규모 언어 모델의 출력을 최적화하여 응답을 생성하기 전에 학습 데이터 소스 외부의 신뢰할 수 있는 지식 베이스(예를 들어 위키피디아)를 참조

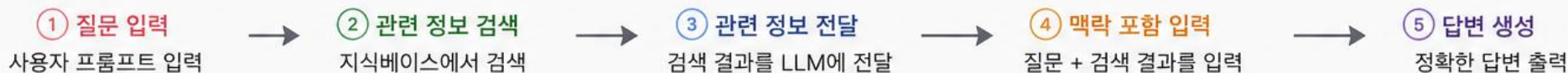
LLM 신뢰성 문제(환각(Hallucination))를 극복하기 위함

RAG (검색 증강 생성) 한눈에 이해하기

RAG는 사용자의 질문에 대해 **관련 정보를 외부 지식베이스에서 검색하고**,
그 정보를 바탕으로 LLM이 **더 정확하고 신뢰할 수 있는 답변을 생성**하는 방식입니다.



흐름 요약





랭체인



LangChain





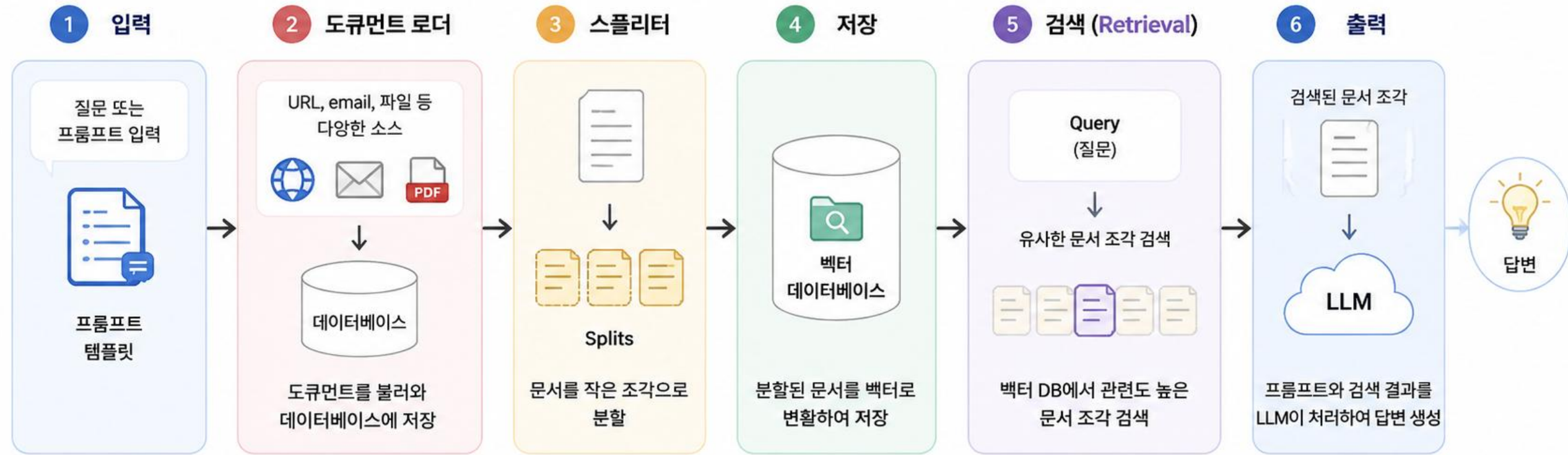
LangChain을 사용하는 이유

이유		설명
1		LLM을 쉽게 연결 OpenAI, Hugging Face, Cohere 등 다양한 LLM API를 손쉽게 호출 가능  OpenAI  Hugging Face  Cohere
2		프롬프트 관리 용이 PromptTemplate 을 사용해 체계적으로 프롬프트 설계 및 재사용 가능
3		메모리 활용 가능 대화 상태를 저장해 대화형 챗봇 개발에 유리
4		벡터 데이터베이스와 통합 가능 FAISS, Pinecone, Weaviate 등과 쉽게 연결하여 장기 기억 구현 가능  FAISS  Pinecone  Weaviate
5		다양한 데이터 소스 활용 PDF, 웹페이지, 데이터베이스 등에서 정보를 읽어와 모델 입력 으로 활용  ...  ...  ...
6		체인(Chain) 기반 워크플로우 LLM 응답을 조합하여 여러 단계의 논리적인 처리 가 가능 1 → 2 → 3 → ... → n
7		에이전트 기반 동적 실행 도구(Tool) 과 연동해 LLM이 동적으로 판단 하고 실행 할 수 있도록 지원  검색  계산  코드 실행 ...  DB 조회



LangChain은 LLM 애플리케이션 개발을 **더 빠르고, 유연하고, 확장성 있게** 만들어 줍니다.

LangChain 동작 흐름 한눈에 보기 ✨



LangChain

위의 모든 단계를 연결하고 관리하여,
질문에 대한 정확하고 신뢰할 수 있는 답변을 생성합니다.



모듈화



유연성



통합성



확장성

FAISS 벡터 DB에는 무엇이 저장될까? (LangChain 기준)

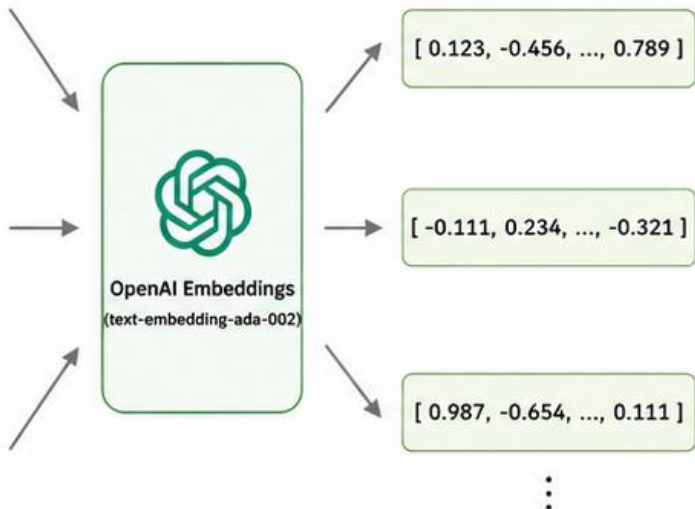
1. 문서 (Documents) 입력

여러 문서를 Document 객체로 준비
(page_content + metadata)



2. 임베딩(Embedding) 생성

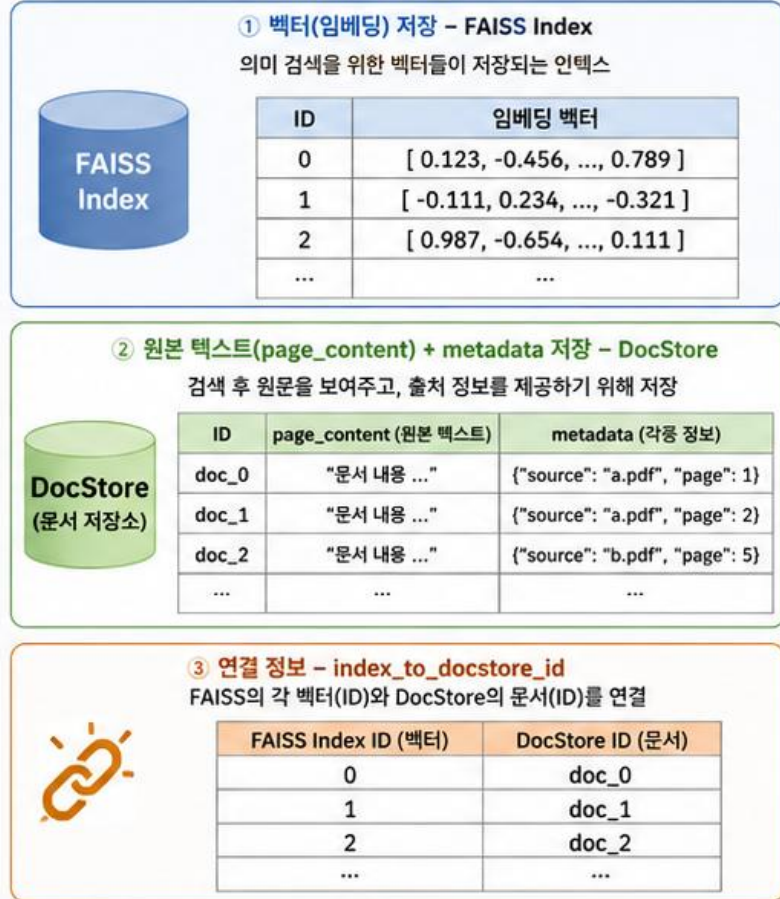
각 문서의 page_content를
임베딩 모델로 변환



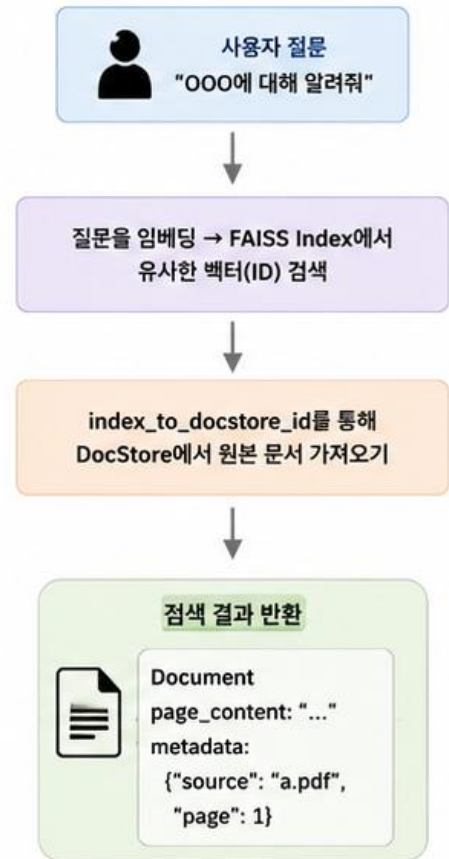
💡 **핵심!**
page_content(텍스트)를 숫자 벡터로 변환하여
의미를 컴퓨터가 이해할 수 있도록 만듭니다.

3. FAISS.from_documents() 실행 후 저장 구조

FAISS 벡터 DB 내부에는 다음 3가지(+ 연결 정보)가 저장됩니다.



4. 검색 시 흐름



정리

- ✓ 1. page_content → 임베딩 벡터 생성 후 FAISS Index에 저장
- ✓ 2. 원본 텍스트(page_content)도 DocStore에 저장
- ✓ 3. metadata(출처, 페이지 번호 등)도 DocStore에 저장
- ✓ 4. 벡터(ID)와 문서(ID)를 연결하는 매핑 정보(index_to_docstore_id) 저장

전체 구조 요약



예시: similarity_search() 결과

```
Document(  
  page_content="문서 내용 ...",  
  metadata={"source": "a.pdf", "page": 1}  
)
```

→ 원문 + metadata 반환!



임베딩(Embedding)이란?

단어나 범주를 컴퓨터가 이해할 수 있는 숫자 벡터로 변환하는 것

원-핫 인코딩처럼 단순히 구분만 하는 것이 아니라, 단어 간의 의미적 유사도를 반영하는 저차원 벡터 표현

1. 원-핫 인코딩(One-Hot Encoding)

단어의 존재 유무만 표현 (의미적 관계를 반영하지 못함)

예시 단어 집합
(VOCABULARY)

사과
바나나
고양이
강아지

원-핫 인코딩

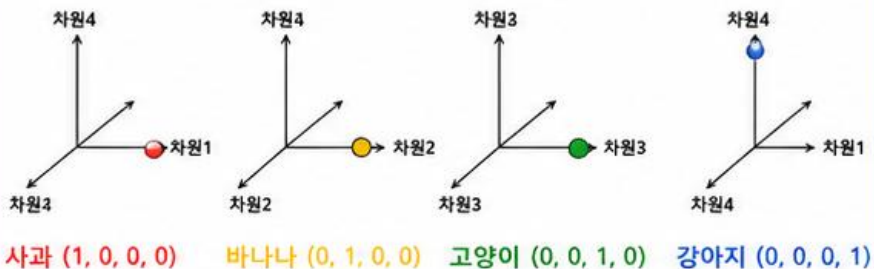
차원 = 단어 집합의 크기(4)

	사과	바나나	고양이	강아지
사과	1	0	0	0
바나나	0	1	0	0
고양이	0	0	1	0
강아지	0	0	0	1

특징

- 차원이 매우 큼 (단어 수가 많아지면 벡터도 매우 길어짐)
- 단어 간 유사도/관계가 전혀 반영되지 않음
- 예) '사과'와 '바나나'는 원-핫 벡터상에서 완전히 다른 벡터 (거리: $\sqrt{2}$)

원-핫 인코딩 벡터의 좌표 표현 (4차원)



두 벡터 간 거리 (예시)

$$\begin{aligned} \text{사과} \leftrightarrow \text{바나나} &= \sqrt{2} \\ \text{사과} \leftrightarrow \text{고양이} &= \sqrt{2} \\ \text{사과} \leftrightarrow \text{강아지} &= \sqrt{2} \end{aligned}$$

2. 임베딩(Embedding)

단어의 의미와 유사도를 반영하는 저차원 벡터

예시 단어 집합
(VOCABULARY)

사과
바나나
고양이
강아지

임베딩 변환

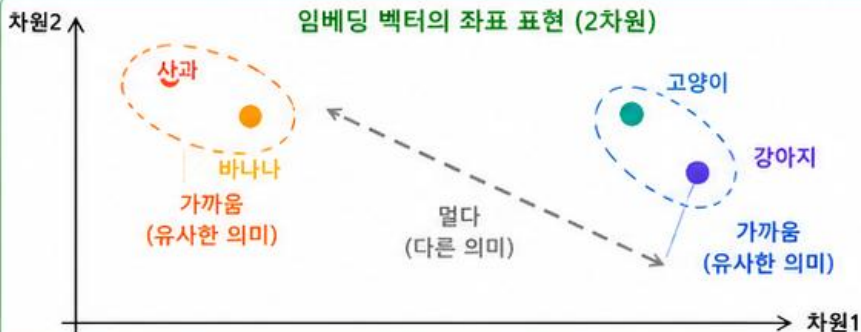
차원 = 임베딩 차원(예: 2차원)

	차원1	차원2
사과	0.71	0.21
바나나	0.69	0.18
고양이	-0.40	0.66
강아지	-0.35	0.61

특징

- 차원이 낮아 효율적 (예: 2, 3, 100, 300 차원 등)
- 단어 간 의미적 유사도(관계)를 벡터 공간의 거리로 표현
- 예) '사과'와 '바나나'는 가까운 벡터, '고양이'와 '강아지'도 가까움

임베딩 벡터의 좌표 표현 (2차원)



두 벡터 간 거리 (예시)

$$\begin{aligned} \text{사과} \leftrightarrow \text{바나나} &= 0.05 \text{ (가까움)} \\ \text{사과} \leftrightarrow \text{고양이} &= 1.35 \text{ (멀다)} \\ \text{고양이} \leftrightarrow \text{강아지} &= 0.07 \text{ (가까움)} \end{aligned}$$



핵심 요약

원-핫 인코딩

= 단어를 구분만 하는 표현



임베딩

= 단어의 의미와 관계까지 표현하는 똑똑한 숫자 벡터

★ 장점

- ✓ 차원 축소로 메모리 효율 증가
- ✓ 단어 간 유사도/관계 학습 가능
- ✓ 머신러닝/딥러닝 성능 향상에 기여

임베딩 벡터의 유사도 구하는 방법 (3가지)

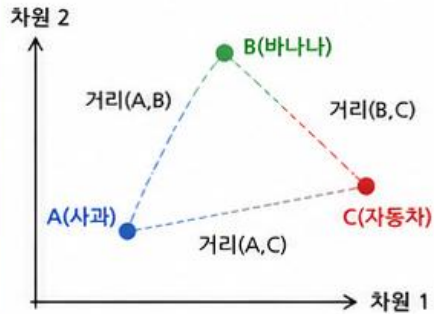


임베딩 벡터는 단어(또는 문장)를 숫자 벡터로 표현한 것입니다.

벡터들 간의 유사도(비슷한 정도)를 측정하는 방법은 여러 가지가 있으며, 대표적으로 **유클리드 거리**, **코사인 유사도**, **내적(점곱) 유사도**가 있습니다.

1. 유클리드 거리 (Euclidean Distance)

두 벡터 사이의 “직선 거리”를 측정하는 방법
거리가 짧을수록 더 유사함 (※ 거리는 0에 가까울수록 유사)



유클리드 거리 계산 공식

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- $\mathbf{x}, \mathbf{y}$: 두 벡터
- n : 벡터의 차원 수

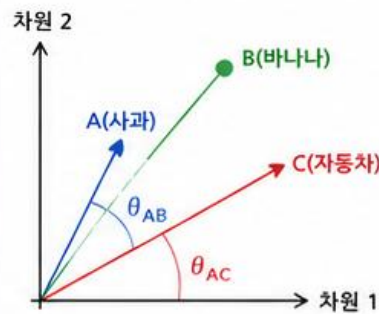
예시 (2차원 임베딩)

단어	벡터 표현	A(사과)와의 거리
A(사과)	[0.1, 0.2]	0
B(바나나)	[0.2, 0.7]	$d = \sqrt{(0.1 - 0.2)^2 + (0.2 - 0.7)^2} = \sqrt{0.26} \approx 0.51$
C(자동차)	[0.9, 0.1]	$d = \sqrt{(0.1 - 0.9)^2 + (0.2 - 0.1)^2} = \sqrt{0.65} \approx 0.81$

결과: A와 B가 더 가까움($0.51 < 0.81$) → ‘사과’와 ‘바나나’가 ‘자동차’보다 더 유사

2. 코사인 유사도 (Cosine Similarity)

두 벡터가 이루는 “각도”의 코사인 값을 이용하는 방법
각도가 작을수록 더 유사함 (※ 값은 -1 ~ 1 사이, 1에 가까울수록 유사)



코사인 유사도 계산 공식

$$\text{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

- $\mathbf{x} \cdot \mathbf{y}$: 내적(dot product)
- $\|\mathbf{x}\|$: 벡터의 크기(노름)

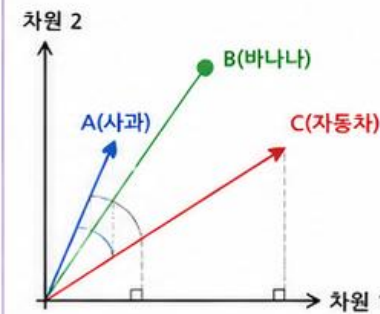
예시 (2차원 임베딩)

단어	벡터 표현	A(사과)와의 코사인 유사도
A(사과)	[0.1, 0.2]	1 (자기 자신)
B(바나나)	[0.2, 0.7]	$\frac{0.1 \times 0.2 + 0.2 \times 0.7}{\sqrt{0.1^2 + 0.2^2} \sqrt{0.2^2 + 0.7^2}} \approx 0.98$
C(자동차)	[0.9, 0.1]	$\frac{0.1 \times 0.9 + 0.2 \times 0.1}{\sqrt{0.1^2 + 0.2^2} \sqrt{0.9^2 + 0.1^2}} \approx 0.24$

결과: A와 B의 방향이 더 비슷함($0.98 > 0.24$) → ‘사과’와 ‘바나나’가 ‘자동차’보다 더 유사

3. 내적(점곱) 유사도 (Dot Product Similarity)

두 벡터를 내적(점곱)하여 얼마나 같은 방향인지와 크기까지 반영하는 방법
값이 클수록 더 유사함 (※ 값의 범위는 $(-\infty, \infty)$)



내적 유사도 계산 공식

$$\text{score}(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i$$

- 값이 클수록 같은 방향 & 큰 크기
- 값이 작거나 음수면 다른 방향

예시 (2차원 임베딩)

단어	벡터 표현	A(사과)와의 내적(점곱) 유사도
A(사과)	[0.1, 0.2]	$0.1 \times 0.1 + 0.2 \times 0.2 = 0.05$ (자기 자신)
B(바나나)	[0.2, 0.7]	$0.1 \times 0.2 + 0.2 \times 0.7 = 0.16$
C(자동차)	[0.9, 0.1]	$0.1 \times 0.9 + 0.2 \times 0.1 = 0.11$

결과: A와 B의 내적 값이 더 큼($0.16 > 0.11$) → ‘사과’와 ‘바나나’가 더 유사



방법
정리
비교

방법	무엇을 측정?	값의 범위	유사할수록?	특징	트랜스포머에서의 사용
유클리드 거리	두 점 사이의 직선 거리	$[0, \infty)$	0에 가까울수록	직관적, 해석 쉬움, 스케일에 영향 받음	직접 사용은 드뭄 (주로 시각화, 클러스터링)
코사인 유사도	두 벡터의 각도	$[-1, 1]$	1에 가까울수록	방향성 비교, 크기(길이)의 영향 적음	일반 임베딩 검색, 문서/단어 유사도, 추천 시스템
내적(점곱) 유사도	두 벡터의 방향 + 크기	$(-\infty, \infty)$	클수록	크기와 방향을 모두 반영	트랜스포머의 핵심! (Self-Attention, $Q \cdot K^T$)

언제 어떤 것을 쓸까?

- **코사인 유사도** : 대부분의 임베딩 검색, 의미적 유사도 비교에 가장 많이 사용
- **내적(점곱) 유사도** : 벡터의 크기 정보도 함께 반영하고 싶을 때, 트랜스포머 내부 연산
- **유클리드 거리** : 공간적 거리 기반 분석, 시각화, 군집화 등에 활용

피타고라스 정리 (Pythagorean Theorem)

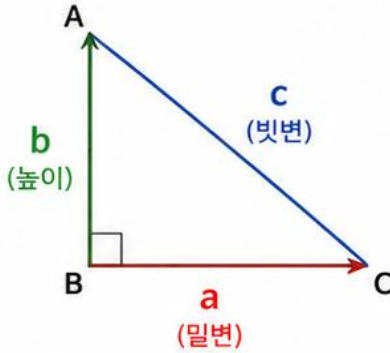


직각삼각형에서 직각을 낀 두 변의 길이 a , b 와 빗변의 길이 c 사이에는 다음 관계가 성립합니다.

$$a^2 + b^2 = c^2$$

a, b : 직각을 낀 두 변 (밑변, 높이)
 c : 빗변 (가장 긴 변)

1. 직각삼각형의 변

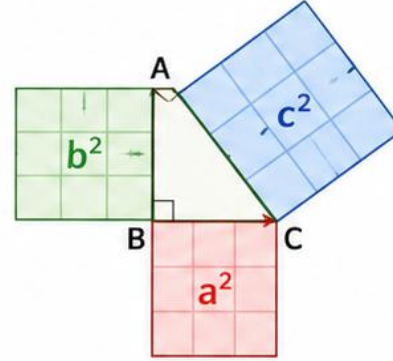


- $\angle B = 90^\circ$ (직각)
- c 가 가장 긴 변
- a, b 는 직각을 낀 두 변

직각이 있는 삼각형에
만 성립해요!



2. 피타고라스 정리의 증명 (도형으로 이해)



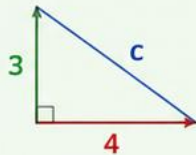
세 변에 정사각형을 만들면
넓이 사이에 다음 관계가
성립합니다.

$$a^2 + b^2 = c^2$$

즉, 작은 두 정사각형의 넓이의
합은 큰 정사각형의 넓이와
같습니다.

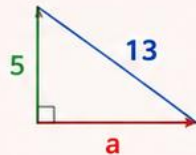
3. 변의 길이를 구하는 방법

(a) 빗변 c 를 구할 때



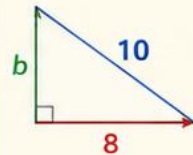
$$\begin{aligned} c^2 &= 3^2 + 4^2 \\ c^2 &= 9 + 16 = 25 \\ c &= 5 \end{aligned}$$

(b) 밑변 a 를 구할 때



$$\begin{aligned} a^2 + 5^2 &= 13^2 \\ a^2 &= 169 - 25 = 144 \\ a &= 12 \end{aligned}$$

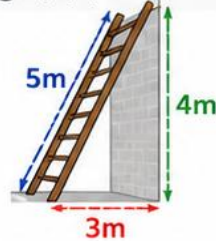
(c) 높이 b 를 구할 때



$$\begin{aligned} 8^2 + b^2 &= 10^2 \\ b^2 &= 100 - 64 = 36 \\ b &= 6 \end{aligned}$$

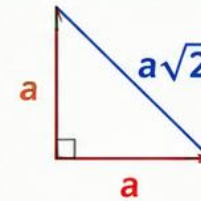
4. 실생활 예시

① 사다리



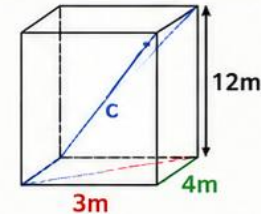
$$\begin{aligned} 3^2 + 4^2 &= 5^2 \\ \text{직각삼각형이 성립!} \end{aligned}$$

② 직각 이등변 삼각형



$$\begin{aligned} a^2 + a^2 &= (a\sqrt{2})^2 \\ \Rightarrow 2a^2 &= 2a^2 \text{ (성립)} \end{aligned}$$

③ 공간에서의 활용



$$\begin{aligned} c^2 &= 3^2 + 4^2 + 12^2 \\ &= 9 + 16 + 144 = 169 \\ c &= 13\text{m} \end{aligned}$$



핵심 정리



직각삼각형에서만 성립



공식

$$a^2 + b^2 = c^2$$



활용

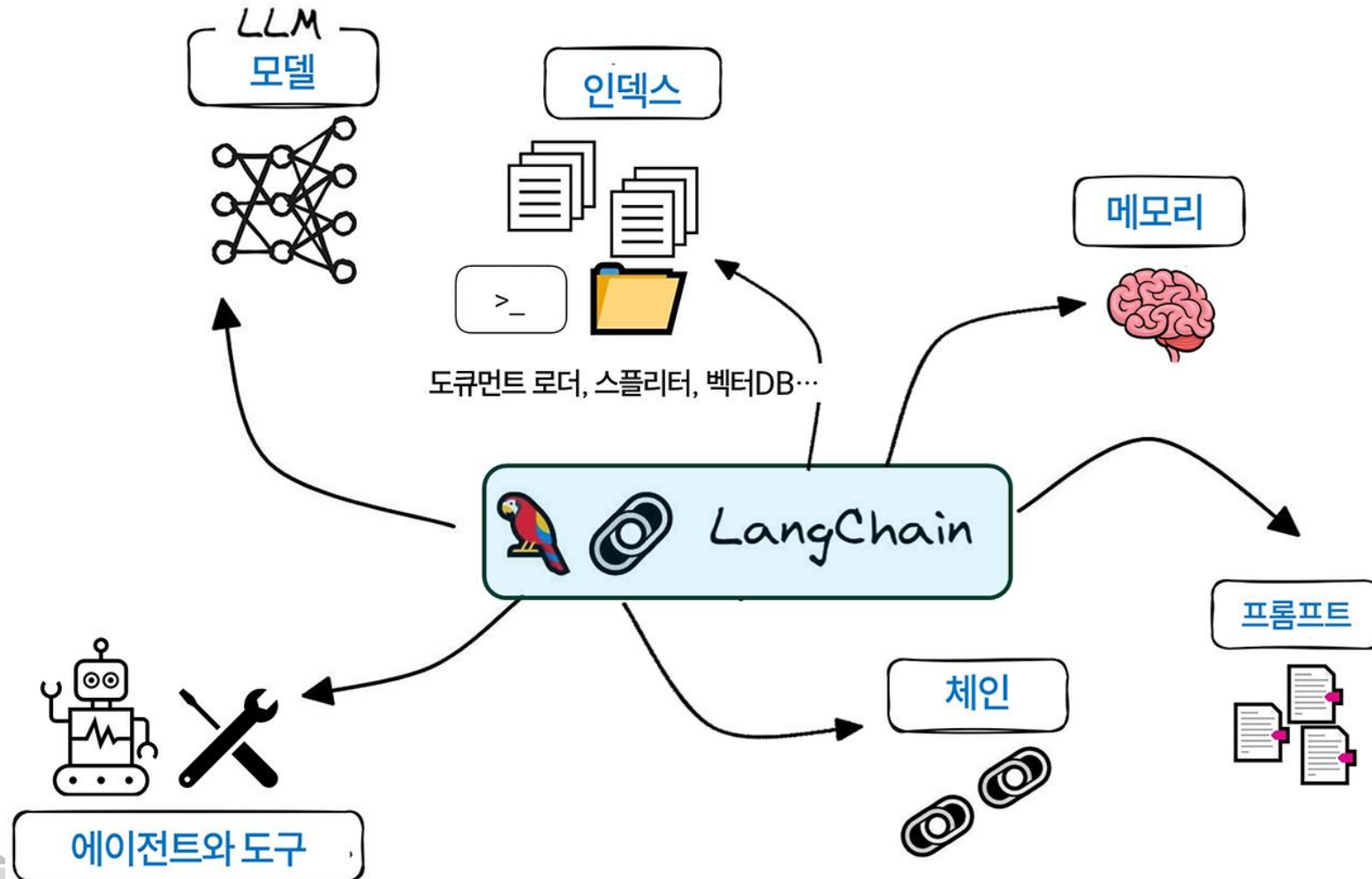
길이 구하기, 거리 계산,
건축, 위치 측정 등



기억법

“작은 둘의 제곱합은
큰 하나의 제곱!”

랭체인 구성도



Streamlit 이란?

Streamlit은 Python만으로 데이터 앱(웹 애플리케이션)을 쉽고 빠르게 만들 수 있는 오픈소스 프레임워크입니다.

주요 특징



Python 기반

Python 스크립트만으로 웹 앱을 개발할 수 있습니다.



빠른 개발

코드를 변경하면 자동으로 앱이 새로고침되어 즉시 반영됩니다.



풍부한 구성 요소

데이터 표시, 차트, 테이블, 입력 폼 등 다양한 위젯을 제공합니다.



손쉬운 배포

몇 번의 명령으로 공유 가능한 웹 앱으로 배포할 수 있습니다.



오픈소스 & 커뮤니티

무료로 사용 가능하며 활발한 커뮤니티의 지원을 받을 수 있습니다.

작동 원리



Python 코드 작성

Streamlit 스크립트 (.py 파일) 작성



실시간 실행

코드가 실행되며 앱이 자동으로 렌더링



웹 브라우저 표시

사용자에게 인터랙티브한 웹 앱을 제공



상호작용 & 업데이트

사용자 입력에 따라 앱이 동적으로 업데이트

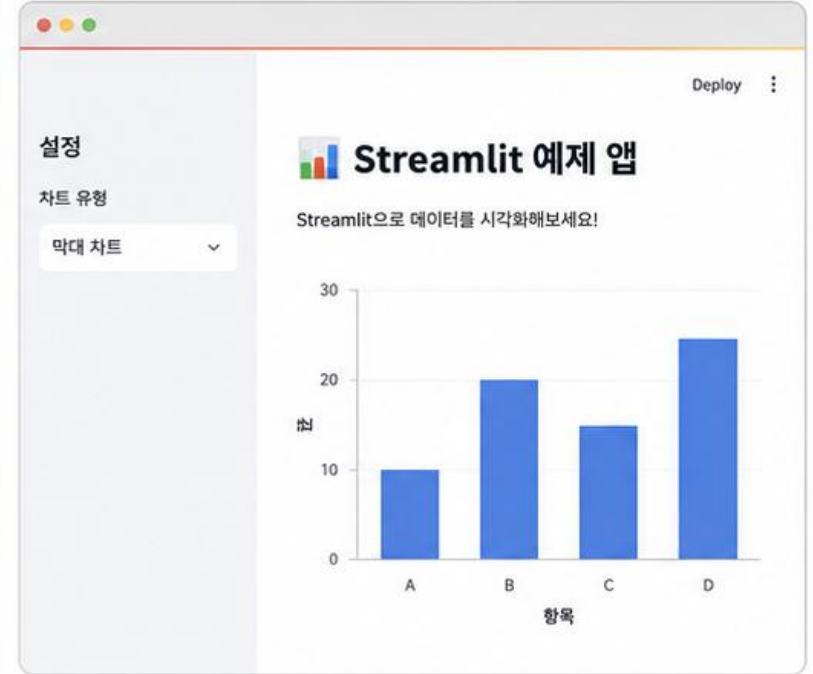
간단한 예제 코드

```
import streamlit as st
import pandas as pd

st.title("📊 Streamlit 예제 앱")
st.write("Streamlit으로 데이터를 시각화해보세요!")
st.sidebar.header("설정")
option = st.sidebar.selectbox("차트 유형", ["막대 차트", "선 차트"])
data = pd.DataFrame({
    '항목': ['A', 'B', 'C', 'D'],
    '값': [10, 20, 15, 25]
})
if option == "막대 차트":
    st.bar_chart(data.set_index('항목'))
else:
    st.line_chart(data.set_index('항목'))
```



실행 결과 예시



활용 분야



데이터 분석 대시보드
데이터를 분석하고 시각화



머신러닝 모델 시연
모델 예측 결과를
인터랙티브하게 제공



내부 도구 & 자동화
사내 업무용 도구나
자동화 앱 구축



교육 & 학습
학습용 앱이나 강의 자료를
쉽게 공유



빠른 프로토타이핑
아이디어를 빠르게 앱으로 만들어
검증하고 공유

배포 방법



streamlit run app.py
로컬에서 실행



Streamlit Community Cloud
또는 클라우드에 배포



공유 가능한 URL로
누구나 접속 가능

배포 명령어

```
streamlit run app.py
# 앱 실행

streamlit share app.py
# 앱 공유

pip install streamlit
# 설치
```



한 줄 요약

Streamlit은 Python만으로 데이터 앱을 쉽고 빠르게 만들고, 공유할 수 있게 도와주는 강력한 도구입니다.

Cloud Run 초고속 배포

코드만 있으면 인프라 걱정 없이, 몇 초 만에 전 세계에 안전하게 배포하세요!



1. Cloud Run 이란?

Google Cloud의 완전관리형(서버리스) 컨테이너 플랫폼입니다.
인프라 관리 없이 컨테이너 애플리케이션을 자동으로 빌드, 배포, 확장합니다.



서버리스

서버 관리, 패치, 용량 계획 불필요



자동 확장

트래픽에 따라 자동으로 확장/축소 (0 → N)



빠른 배포

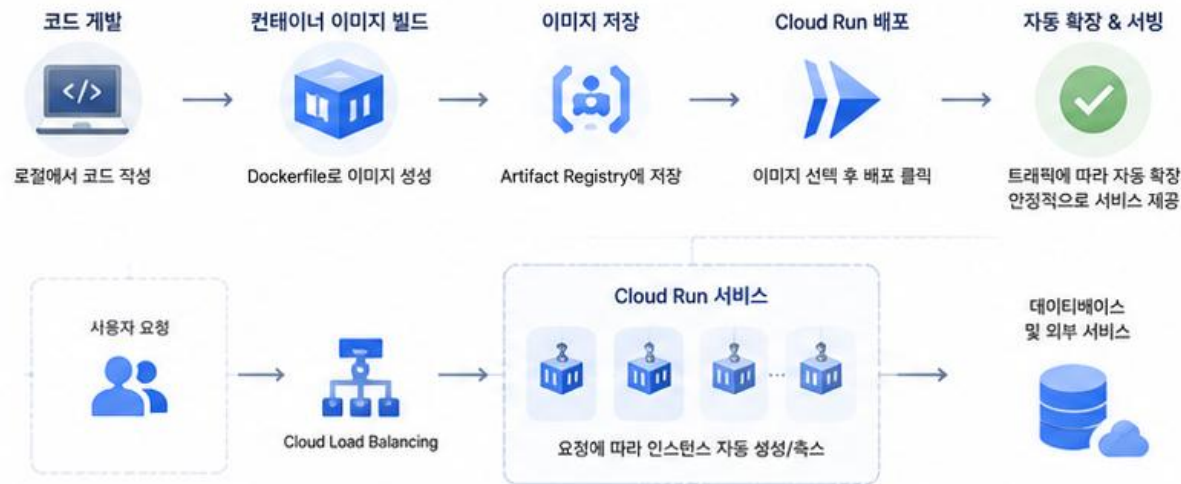
컨테이너 이미지 푸시 후 몇 초 만에 배포 완료



전 세계 제공

전 세계 엣지에서 빠르고 안전하게 서비스 제공

2. Cloud Run 배포 흐름



3. 초고속 배포의 장점



몇 초 만에 배포 완료
이미지 업로드 후 클릭 한 번으로
서비스 즉시 실행



트래픽 급증에도 자동 대응
자동 확장으로 성능 저하 없이 안정적 운영



비용 절감
요청이 없으면 0으로 축소되어
사용한 만큼만 비용 지불



운영 부담 최소화
서버, 인프라, OS 관리 없이
개발에만 집중 가능



전 세계 빠른 응답
Google 글로벌 네트워크를 통한
낮은 지연시간 및 높은 가용성

4. 배포 방법 (예시)

```
# 1. 이미지 빌드
docker build -t gcr.io/PROJECT_ID/my-app .
# 2. 이미지 푸시
docker push gcr.io/PROJECT_ID/my-app
# 3. Cloud Run 배포
gcloud run deploy my-app \
  --image gcr.io/PROJECT_ID/my-app \
  --region asia-northeast3 \
  --platform managed \
  --allow-unauthenticated
```

5. 주요 기능



동시 요청 처리
높은 동시성용
지원하는 확장성



리버전 관리
트래픽 분할 및
롤백이 쉬운 배포



환경 변수/비밀 관리
Secret Manager와
연동



VPC 연결
안전한 내부 네트워크
역세스 지원



Cloud Monitoring
로깅, 모니터링,
알림 제공



IAM 통합
세밀한 권한 제어로
안전한 운영

6. 사용 사례



웹 애플리케이션
& API



모바일 백엔드
서비스



AI/ML 모델
서빙



이벤트 기반 처리
작업



아이크로서비스
아키텍처



한 줄 요약

Cloud Run은 서버 관리 없이 컨테이너를 몇 초 만에 배포하고, 트래픽에 맞춰 자동 확장하여 안정적으로 서비스를 운영할 수 있게 해주는 초고속 서버리스 플랫폼입니다.